

Observability & Monitoring — The Complete Guide

Logging → Metrics → Tracing → Alerting, for Java Full Stack Developers

Log4j/SLF4J • ELK/Kibana • Splunk (with real SPL queries) • Prometheus/Grafana •
OpenTelemetry/Zipkin/Jaeger

Spring Boot Actuator/Micrometer • Azure Monitor/Azure DevOps • Real Production
Troubleshooting

From "Hello World" console logs to full enterprise observability — simple terms, real dashboards, real queries

Table of Contents

1. Why Logging Is Needed — Starting with a Simple "Hello World" Application
2. Before Logging Frameworks — What Developers Used to Do
3. Introduction to Log4j, SLF4J and Logback in Java
4. Log Levels Explained (TRACE, DEBUG, INFO, WARN, ERROR)
5. What Is Observability? Monitoring vs Observability
6. The Three Pillars of Observability — Logs, Metrics, Traces
7. Golden Signals — Latency, Traffic, Errors, Saturation
8. Types of Logs — What We Log and What We Monitor
9. Structured Logging, Correlation ID, Trace ID & Span ID
10. Centralized Logging & Log Aggregation — Why It's Mandatory
11. ELK Stack Deep Dive (Elasticsearch, Logstash, Kibana)
12. Kibana Dashboards — Visual Examples & Real Queries
13. Splunk Deep Dive — Architecture, Indexing, Forwarders
14. Splunk Search Processing Language (SPL) — Real Queries
15. Splunk Dashboards — Visual Examples
16. ELK vs Splunk vs Kibana — Full Comparison
17. Migrating from Browser/Console Logs to Splunk or ELK (Real Migration Path)
18. Application, Infrastructure & Business Metrics
19. Prometheus & Grafana Deep Dive with Dashboards
20. Spring Boot Observability — Actuator & Micrometer
21. Distributed Tracing — OpenTelemetry, Zipkin, Jaeger
22. Monitoring Types — JVM, CPU, Memory, Disk, Network, Database, API
23. APM Tools — Dynatrace, AppDynamics, New Relic
24. Alerting — SLA, SLO, SLI, Alertmanager, PagerDuty
25. Azure Monitor & Azure DevOps Integration for Logging and Monitoring
26. Real-Time Production Troubleshooting (Response Time, Memory Leak, CPU, Thread Dump, GC, Slow SQL)
27. Real Organization Setup — End to End (Our Application)
28. Common Observability Issues, Problems & Solutions
29. Interview Questions & Answers (Basic → Advanced → Scenario Based)
30. Final Summary & Cheat Sheet

1. Why Logging Is Needed — Starting with a Simple "Hello World" Application

1.1 The Simplest Possible Program

```
public class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

This works fine — you run it, you see "Hello, World!" in the console, done. But real applications aren't run once by a developer watching the console. They run 24/7, unattended, on remote servers, handling thousands of requests, and something eventually goes wrong when nobody is watching the screen.

1.2 What Happens When This Program Grows Up

```
public class OrderProcessor {  
    public static void main(String[] args) {  
        System.out.println("Starting order processing...");  
        try {  
            processOrder(1001);  
        } catch (Exception e) {  
            System.out.println("Something went wrong!"); // this tells you NOTHING useful  
        }  
    }  
}
```

If this ran on a production server last night at 2 AM and failed, all you have is "Something went wrong!" printed into a console window nobody was watching — it's already scrolled away or the terminal was closed. This is the exact problem logging solves.

1.3 The Core Reasons We Need Logging

Reason	Simple Explanation
Nobody watches the console 24/7	Logs written to a file/system persist even when no one is looking, so you can check them later
Understand what happened before a crash	A log trail shows the sequence of events leading up to a failure
Debug issues that only happen in production	You can't attach a debugger to a live production server — logs are often your only window into what happened
Track business events	"Order #1001 placed by customer #55 at 10:42 AM" — useful for audits, support, and analytics, not just bugs
Security & compliance	Who accessed what, when — required for audits in banking/healthcare

Performance analysis

Logs with timestamps reveal how long each step took

1.4 From print() to Real Logging — The Journey

The Evolution of "Telling the Developer What's Happening"

`System.out.println()`

Log4j / SLF4J / Logback (files, levels)



Centralized Logging (ELK / Splunk)



Full Observability (Logs + Metrics + Traces)

This entire guide follows exactly this journey — starting simple, and building up to the full enterprise observability stack real organizations use in production, including the one your organization uses today (Splunk + Azure DevOps).

2. Before Logging Frameworks — What Developers Used to Do

2.1 The print() / System.out.println() Era

Before dedicated logging frameworks existed, Java developers simply used `System.out.println()` (or `System.err.println()` for errors) scattered throughout the code. This "worked" for tiny programs but broke down fast in real applications.

2.2 Problems With Using println() in Real Applications

Problem	Why It Hurts in Production
No log levels	Can't say "only show me errors" — every println prints unconditionally, cluttering output
No timestamps by default	You don't know exactly when something happened without manually adding it every time
Nowhere persistent to look	Console output disappears when the terminal closes or the server restarts
No way to turn off verbose logs in production	You'd have to comment out code and redeploy just to reduce noise
Can't easily route logs elsewhere	No built-in way to send logs to a file, a database, or a remote log server
Poor performance at scale	Uncontrolled, unbuffered console writes across many threads slow down high-traffic applications
No structure	Free-form text is hard to search/filter/parse reliably at scale

2.3 A Real Example of the Old Way

```
// Old-style manual "logging" — common in legacy codebases before Log4j adoption
public class LegacyPaymentService {
    public void processPayment(double amount) {
        System.out.println "[" + new java.util.Date() + "] Processing payment: " + amount);
        try {
            // ... payment logic
        } catch (Exception e) {
            System.out.println "[" + new java.util.Date() + "] ERROR: " + e.getMessage());
            e.printStackTrace(); // prints directly to console, uncontrolled
        }
    }
}
```

Every developer manually formatted timestamps differently, there was no consistent way to filter by severity, and turning this "off" for production meant physically deleting or commenting out print statements — a real, common pain point that led to the creation of dedicated logging frameworks like Log4j.

3. Introduction to Log4j, SLF4J and Logback in Java

3.1 Why Log4j Was Created

Log4j (created in 1999) was one of the first Java logging frameworks to solve exactly the problems in Section 2 — configurable log levels, multiple output destinations ("appenders": console, file, database, remote server), and configurable message formatting, all without changing code.

3.2 The Modern Java Logging Landscape

Component	Role
SLF4J (Simple Logging Facade for Java)	A common API /facade your code depends on — you write code against SLF4J, and the actual logging framework underneath can be swapped without changing your code
Logback	The most common modern implementation used underneath SLF4J — Spring Boot's default logging framework
Log4j2	The modern successor to the original Log4j — a full-featured framework, an alternative to Logback, often chosen for higher performance async logging

Interview tip: SLF4J is not a logging implementation itself — it's a facade. This means you can code entirely against the SLF4J API and switch the actual underlying engine (Logback vs Log4j2) via dependencies alone, with zero code changes.

3.3 Real Code — Using SLF4J (What You Actually Write in Spring Boot)

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class PaymentService {
    private static final Logger log = LoggerFactory.getLogger(PaymentService.class);

    public void processPayment(double amount) {
        log.info("Processing payment of amount={}", amount);
        try {
            // ... payment logic
        } catch (Exception e) {
            log.error("Payment processing failed for amount={}", amount, e); // e = full stack trace logged
        }
    }
}

// With Lombok, this becomes even shorter:
@Slf4j
public class PaymentService {
    public void processPayment(double amount) {
        log.info("Processing payment of amount={}", amount);
    }
}
```

3.4 Log4j2 Configuration Example (log4j2.xml)

```
<Configuration status="WARN">
  <Appenders>
    <Console name="Console" target="SYSTEM_OUT">
      <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %logger{36} - %msg%n"/>
    </Console>
    <RollingFile name="File" fileName="logs/app.log" filePattern="logs/app-%d{yyyy-MM-dd}-%i.log.gz">
      <PatternLayout pattern="%d{ISO8601} [%t] %-5level %logger{36} - %msg%n"/>
      <Policies>
        <TimeBasedTriggeringPolicy/>
        <SizeBasedTriggeringPolicy size="50MB"/>
      </Policies>
      <DefaultRolloverStrategy max="10"/>    <!-- keep last 10 rotated files -->
    </RollingFile>
  </Appenders>
  <Loggers>
    <Logger name="com.bank" level="INFO" additivity="false">
      <AppenderRef ref="Console"/>
      <AppenderRef ref="File"/>
    </Logger>
    <Root level="WARN">
      <AppenderRef ref="Console"/>
    </Root>
  </Loggers>
</Configuration>
```

This configuration says: log everything INFO and above from `com.bank` package code to both console and a rotating file (max 50MB per file, gzip old ones, keep last 10) — but only WARN and above for all other (third-party library) code, keeping noise down.

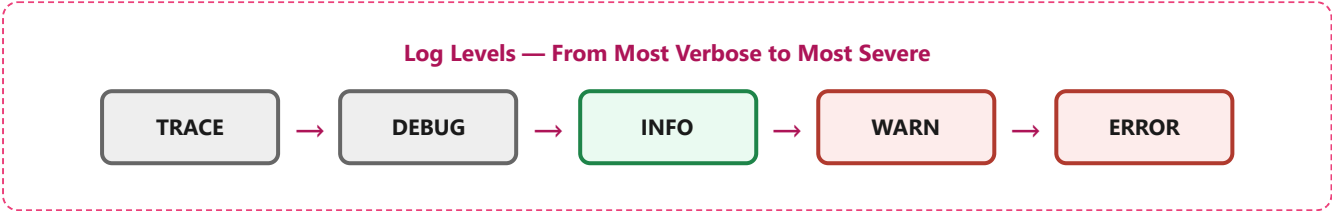
3.5 Spring Boot's Default — Logback (application.yml)

```
logging:
  level:
    root: WARN
    com.bank: INFO
  file:
    name: logs/accounts-service.log
  pattern:
    file: "%d{ISO8601} [%X{traceId}] %-5level %logger{36} - %msg%n"
```

Notice `%X{traceId}` — this pulls a value from the SLF4J MDC (Mapped Diagnostic Context), which is exactly how Correlation IDs and Trace IDs get automatically included in every log line (full detail in Section 9).

4. Log Levels Explained (TRACE, DEBUG, INFO, WARN, ERROR)

4.1 The Standard Log Level Hierarchy



4.2 What Each Level Means and When to Use It

Level	When to Use	Real Example
TRACE	Extremely detailed, step-by-step internal flow — rarely enabled, even in dev	"Entering method calculateInterest() with params: ..."
DEBUG	Useful detail for developers diagnosing an issue, too noisy for production by default	"SQL query built: SELECT * FROM account WHERE id=?"
INFO	Normal but significant business/application events — the default production level	"Order #1001 created successfully for customer #55"
WARN	Something unexpected happened but the application recovered/continued	"Retrying payment gateway call, attempt 2 of 3"
ERROR	Something failed and needs attention — an operation could not complete	"Failed to process payment for order #1001: connection timeout"

4.3 Setting Levels Per Environment (Real Practice)

Environment	Typical Root Level	Why
Local Development	DEBUG	Developers want maximum visibility while building/debugging features
QA/Staging	INFO or DEBUG (temporarily)	Enough detail to diagnose test failures without overwhelming volume
Production	INFO (WARN for noisy third-party libraries)	DEBUG/TRACE in prod at scale generates huge log volume, cost, and can leak sensitive data

Real mistake to avoid: Leaving DEBUG logging enabled in production for a high-traffic service can silently fill up disk space or massively increase your Splunk/ELK ingestion costs — always confirm log levels before a production release, and use dynamic log level changes (Actuator's `/loggers` endpoint) instead of full redeloys when you temporarily need more detail.

4.4 Changing Log Levels at Runtime (Spring Boot Actuator)

```
# View current level
GET /actuator/loggers/com.bank.accounts

# Temporarily bump to DEBUG without redeploying – great for live troubleshooting
POST /actuator/loggers/com.bank.accounts
{ "configuredLevel": "DEBUG" }
```

5. What Is Observability? Monitoring vs Observability

5.1 What Is Observability?

Observability is the ability to understand what's happening **inside** a system just by examining its external outputs — logs, metrics, and traces — even for problems you never anticipated in advance. It answers "why" something is happening, not just "that" something is happening.

5.2 What Is Monitoring?

Monitoring is watching a predefined set of metrics/dashboards/alerts for known failure conditions you already anticipated — "CPU > 90%," "error rate > 5%." It tells you **that** something is wrong.

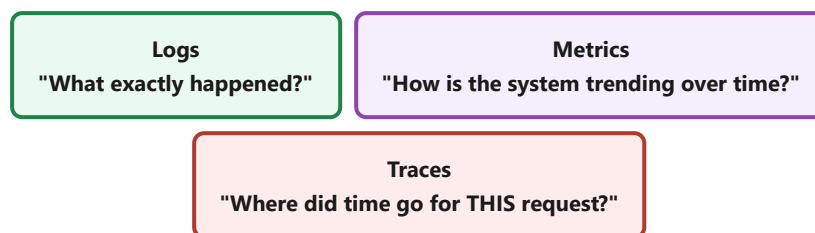
5.3 Monitoring vs Observability — Side by Side

Aspect	Monitoring	Observability
Approach	Watches known, predefined metrics/dashboards	Lets you explore and ask new questions about unknown issues
Answers	"Is the system healthy right now?"	"Why exactly is this specific request slow?"
Good for	Known failure modes (disk full, CPU spike)	Novel, first-time-seen production issues
Built from	Dashboards and alert rules	Logs + Metrics + Traces combined, richly correlated by IDs
Relationship	Monitoring is a subset of Observability — you need good telemetry data (observability) to build good monitoring on top of it	

Interview one-liner: "Monitoring tells you something is broken; observability helps you figure out why — you can have monitoring without full observability, but you can't have real observability without collecting the data monitoring depends on."

6. The Three Pillars of Observability — Logs, Metrics, Traces

The Three Pillars — What Each One Answers



6.1 Logs

Discrete, timestamped, detailed event records — an error message, a business event, a warning. Best for deep-diving into **one specific incident or request**.

6.2 Metrics

Numeric measurements aggregated over time — request count, average latency, memory usage percentage. Best for **trends, dashboards, and alerting thresholds**; cheap to store even at high volume since they're pre-aggregated numbers, not full text.

6.3 Traces

The end-to-end path of a single request as it flows through multiple services/components, broken into timed "spans." Best for finding **exactly which hop** in a multi-service call chain is slow or failing.

6.4 How They Work Together — Real Example

Using All Three Pillars Together to Debug an Incident

1. Grafana metric dashboard shows: Accounts Service p99 latency spiked to 3s



2. Open Jaeger, find a slow trace from that time window

↓ trace shows one span (Cards Service call) took 2.8s of the 3s

3. Search Kibana/Splunk logs filtered by that trace ID — find the exact error/warning logged by Cards Service at that moment

7. Golden Signals — Latency, Traffic, Errors, Saturation

Popularized by Google's SRE book, the "Four Golden Signals" are the minimum set of metrics you should monitor for any user-facing service if you can only pick a few.

Signal	What It Measures	Real Example Metric
Latency	Time taken to serve a request	p50/p95/p99 response time of <code>POST /accounts/transfer</code>
Traffic	Demand being placed on the system	Requests per second on the API Gateway
Errors	Rate of failed requests	Percentage of 5xx responses from Cards Service
Saturation	How "full" the system's resources are	CPU/memory/thread pool utilization, DB connection pool usage

Real scenario: "Our Grafana dashboard for Accounts Service has exactly these four panels at the very top — any on-call engineer can glance at it for 5 seconds and know if the service is healthy, overloaded, or actively failing, before digging into anything deeper."

8. Types of Logs — What We Log and What We Monitor

8.1 Categories of Logs

Log Type	What It Contains	Real Example
Application Logs	Business logic events, errors, warnings from your own code	"Loan application LN-4521 approved for customer 8823"
Access/Request Logs	Every incoming HTTP request — method, path, status, duration	"GET /accounts/1001 → 200 OK in 42ms"
Error/Exception Logs	Stack traces and failure details	"NullPointerException at AccountService.java:82"
Audit Logs	Who did what, when — for compliance/security	"User admin_raj updated card limit for card 8890 from 50000 to 100000"
Security Logs	Authentication attempts, authorization failures, suspicious activity	"Failed login attempt for user jsmith from IP 203.0.113.44"
Infrastructure/System Logs	OS, container runtime, Kubernetes events	"Pod cards-service-7f9c OOMKilled"
Database Logs	Slow queries, connection errors, deadlocks	"Query exceeded 5000ms: SELECT * FROM transaction WHERE ..."
Integration/Third-Party Logs	Calls to external systems (payment gateways, credit bureaus)	"Card network API responded with timeout after 10s"

8.2 What We Actually Monitor (Not Just Log)

Category	What We Watch
Availability	Is the service up and responding to health checks?
Performance	Response time percentiles, throughput
Errors	Error rate/count, by type and endpoint
Resource usage	CPU, memory, disk, network, thread pools, DB connections
Business KPIs	Orders placed/hour, loan approvals/day, failed payment percentage
Security events	Failed logins, unusual access patterns, privilege escalations

What NOT to log: passwords, full credit card numbers, JWT tokens, national ID numbers, or other sensitive PII/PCI data in plain text — mask/redact these before logging (e.g., log only the last 4 digits of a card number) to stay compliant with PCI-DSS/GDPR and avoid a serious security incident if logs are ever exposed.

9. Structured Logging, Correlation ID, Trace ID & Span ID

9.1 Unstructured vs Structured Logging

```
# Unstructured (plain text) – hard to search/filter reliably at scale
2026-09-30 10:42:13 INFO Order 1001 created for customer 55, amount 4500.00

# Structured (JSON) – every field is queryable, filterable, aggregatable
{"timestamp":"2026-09-30T10:42:13Z","level":"INFO","service":"orders-service",
 "traceId":"a1b2c3d4","message":"Order created","orderId":1001,"customerId":55,"amount":4500.00}
```

Structured (usually JSON) logs let tools like Splunk/ELK index each field separately — you can query `customerId=55 AND level=ERROR` directly, instead of writing fragile text-pattern searches across free-form messages.

9.2 Real Spring Boot Structured Logging Setup (Logback JSON Encoder)

```
<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
</dependency>

<!-- logback-spring.xml -->
<appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
  <encoder class="net.logstash.logback.encoder.LogstashEncoder">
    <includeMdcKeyName>traceId</includeMdcKeyName>
    <includeMdcKeyName>spanId</includeMdcKeyName>
  </encoder>
</appender>
```

9.3 What Is a Correlation ID?

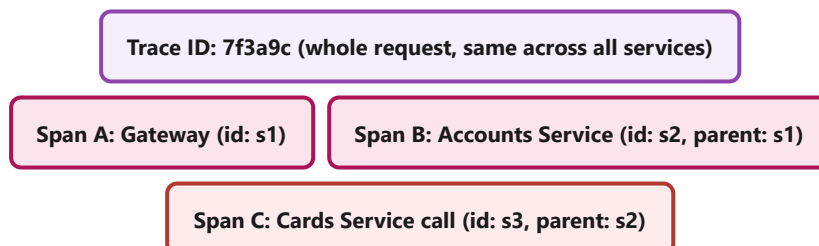
A Correlation ID is a unique identifier generated at the very start of a request (usually at the API Gateway or the first service that receives it) and passed along in every downstream call's headers/logs — letting you find **every log line related to one user's request** across every service it touched, just by searching for that one ID.

```
// Servlet Filter generating/propagating a correlation ID
@Component
public class CorrelationIdFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String correlationId = req.getHeader("X-Correlation-Id");
        if (correlationId == null) correlationId = UUID.randomUUID().toString();
        MDC.put("correlationId", correlationId);    // now available to every log line via %X{correlationId}
        res.setHeader("X-Correlation-Id", correlationId);
        try {
            chain.doFilter(req, res);
        } finally {
            MDC.clear();    // always clean up MDC to avoid leaking into the next request on the same thread
        }
    }
}
```

9.4 Trace ID & Span ID (Distributed Tracing Terms)

A **Trace ID** is essentially a more formal, standardized version of a Correlation ID used specifically by distributed tracing systems (OpenTelemetry, Zipkin, Jaeger) — it identifies the entire end-to-end request across all services. A **Span ID** identifies one specific unit of work within that trace (e.g., "the Cards Service's handling of this request" is one span, "the SQL query it ran" could be a child span).

Trace ID vs Span ID



Real scenario: "A customer reports 'my transfer failed.' We ask for the correlation ID from the error screen (or find it via their customer ID + timestamp), search Splunk for that exact ID, and instantly see every log line from Gateway → Accounts Service → Cards Service → Kafka for that one specific request — no guessing which of 50 million daily log lines are relevant."

10. Centralized Logging & Log Aggregation — Why It's Mandatory

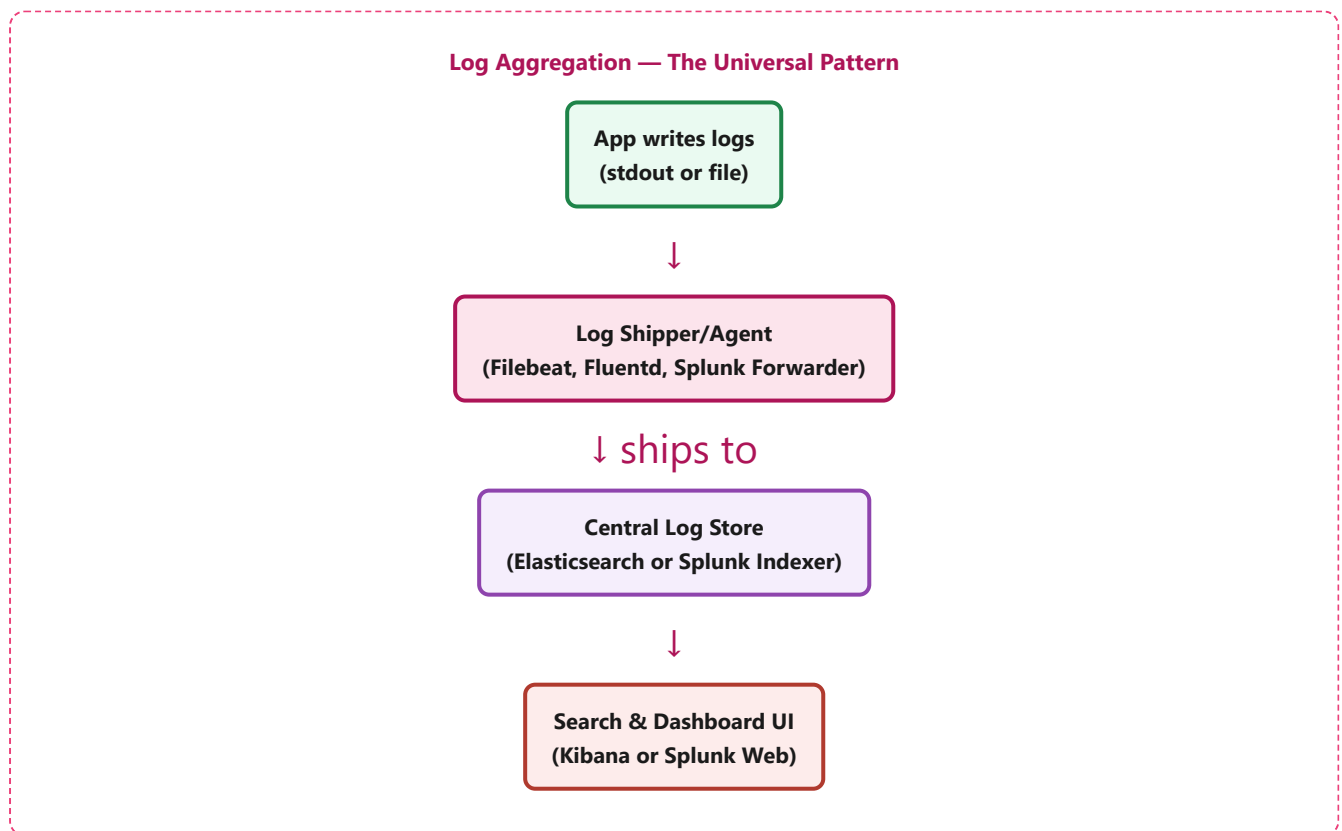
10.1 The Problem Without Centralized Logging

With 20 microservices running across dozens of Kubernetes pods, logs sitting only on each pod's local disk are nearly useless — pods are ephemeral (deleted/rescheduled constantly), so their logs vanish, and manually SSHing/kubectl-exec-ing into each pod to grep logs one by one doesn't scale at all.

10.2 What Centralized Logging Provides

- One searchable place for logs from every service, every pod, every environment.
- Logs survive even after the pod that generated them is gone.
- Powerful search/filter across huge volumes (millions of log lines/day) in seconds.
- Dashboards and alerts built directly on log data (e.g., alert if "ERROR" count spikes).
- Long-term retention for audits/compliance, independent of any single server's disk.

10.3 General Log Aggregation Architecture (Tool-Agnostic)



This same 4-layer pattern (App → Shipper → Store → UI) is exactly how both ELK and Splunk work internally — once you understand one, the other is easy to map onto the same mental model, just with different product names.

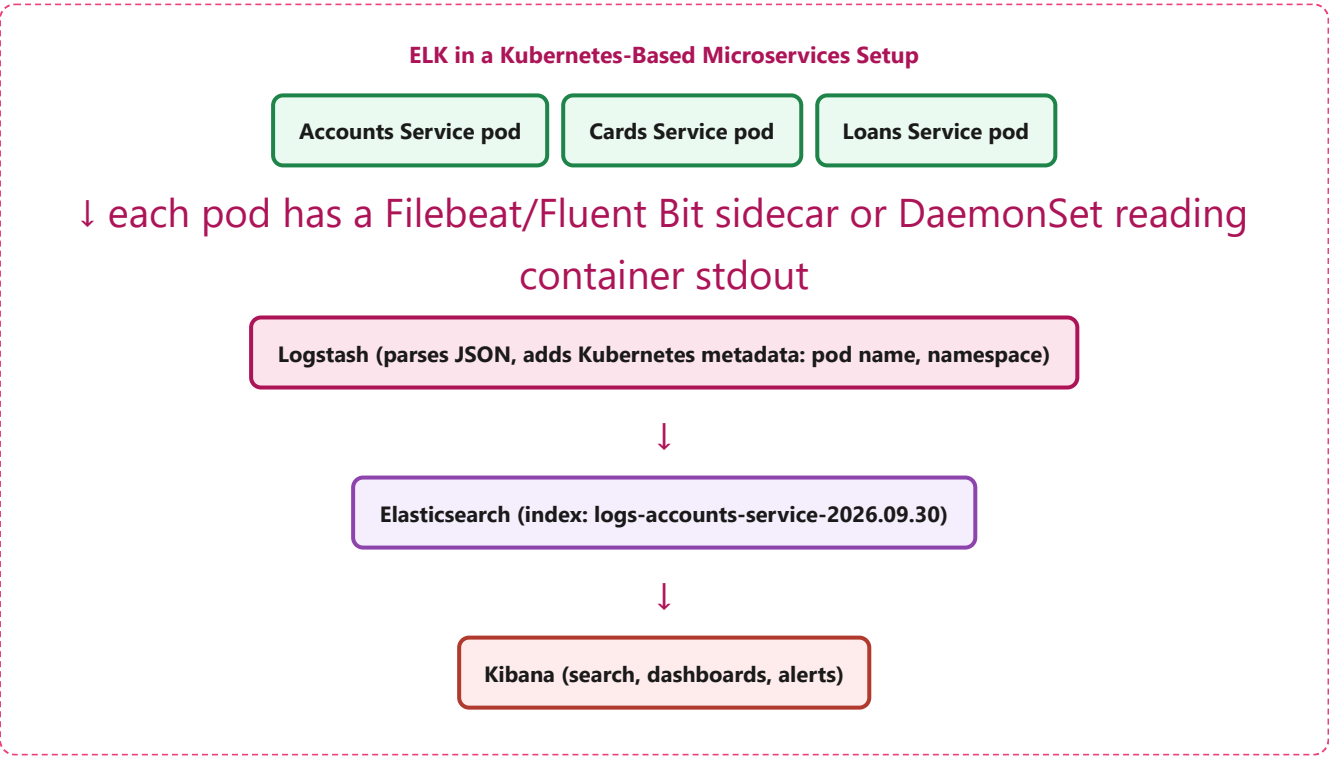
11. ELK Stack Deep Dive (Elasticsearch, Logstash, Kibana)

11.1 The Three Components

Component	Role
Elasticsearch	A distributed search & analytics engine — stores logs as indexed JSON documents, enables fast full-text and structured search
Logstash	A data processing pipeline — ingests raw logs, parses/transforms/enriches them, then sends them to Elasticsearch
Kibana	The web UI — search logs, build visualizations and dashboards on top of Elasticsearch data

Many modern setups replace Logstash with the much lighter-weight **Filebeat** (a "Beat") for simple log shipping, or use **Fluentd/Fluent Bit** (common in Kubernetes) — the combination is then sometimes called "EFK" instead of "ELK."

11.2 Full ELK Flow — Real Example



11.3 Elasticsearch Core Concepts

Concept	Meaning
Index	A collection of documents (like a database/table), often one per service per day (e.g., <code>logs-accounts-2026.09.30</code>)
Document	One single log entry, stored as JSON
Field	One property of the document (level, message, traceId, customerId)

Shard	A horizontal partition of an index, spread across nodes for scalability
Query DSL	Elasticsearch's JSON-based query language for searching documents

11.4 Real Elasticsearch Query DSL Example

```
GET /logs-accounts-service-*/_search
{
  "query": {
    "bool": {
      "must": [
        { "match": { "level": "ERROR" } },
        { "match": { "traceId": "a1b2c3d4" } }
      ],
      "filter": [
        { "range": { "timestamp": { "gte": "now-1h" } } }
      ]
    }
  }
}
```

11.5 A Basic Logstash Pipeline Config

```
input {
  beats { port => 5044 }
}
filter {
  json { source => "message" }
  mutate { add_field => { "environment" => "prod" } }
}
output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "logs-%{[service]}-%{+YYYY.MM.dd}"
  }
}
```

12. Kibana Dashboards — Visual Examples & Real Queries

12.1 Kibana Query Language (KQL) — Real Search Examples

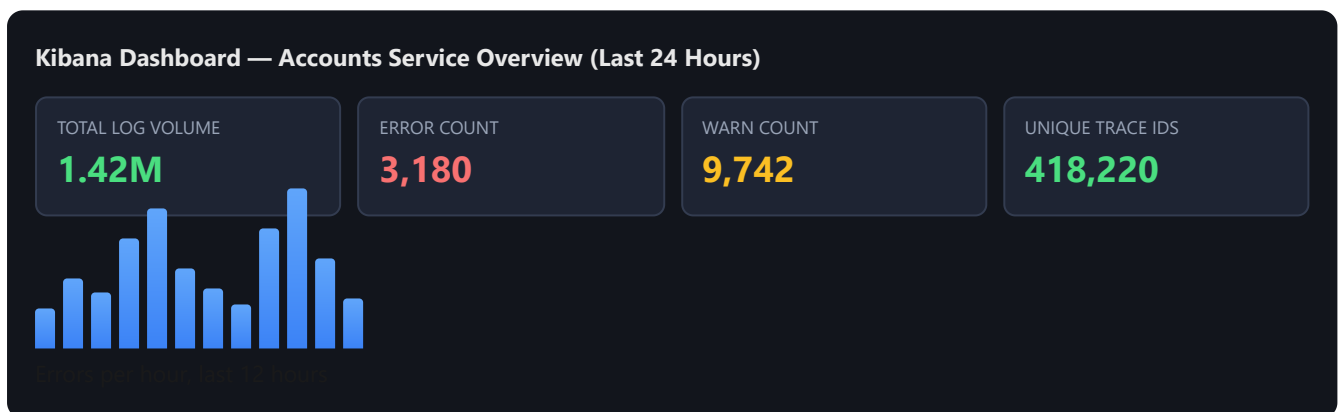
```
# Find all ERROR logs from Cards Service in the last hour
service: "cards-service" and level: "ERROR"

# Find everything for one specific customer's failed transactions
customerId: 55 and message: "**failed*"

# Find slow requests (custom field "durationMs" logged by the app)
service: "accounts-service" and durationMs > 2000

# Find all logs for one traced request across every service
traceId: "a1b2c3d4"
```

12.2 Example Kibana Dashboard — Visual Mockup



12.3 Common Kibana Dashboard Panels Real Teams Build

- **Log volume over time** — a line/bar chart showing overall traffic and spotting unusual spikes.
- **Top error messages** — a data table aggregating the most frequent error strings, to spot recurring issues.
- **Errors by service** — a pie/bar chart comparing which microservice is generating the most errors.
- **Response time heatmap** — visualize latency distribution over time.
- **Geographic/customer impact map** — for customer-facing incidents, showing how many unique customers were affected.

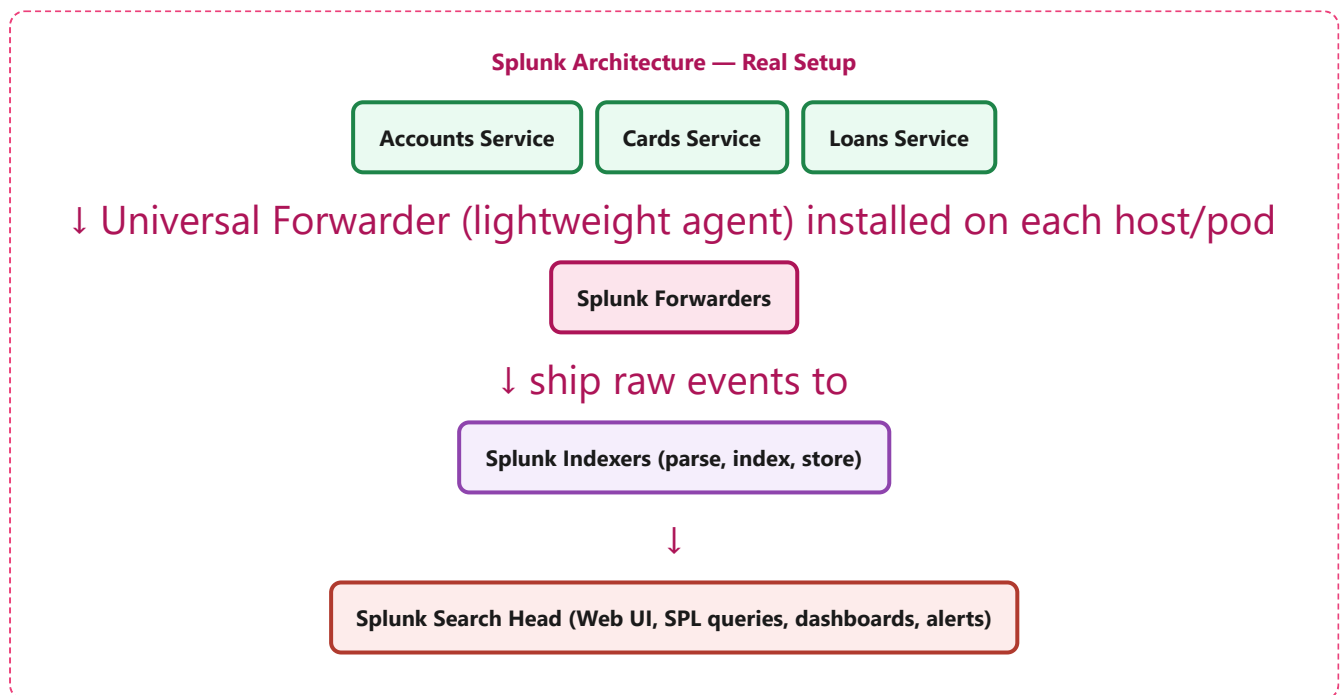
Real scenario: "During a production incident, the on-call engineer opens the pre-built 'Live Incident' Kibana dashboard, filters by the last 15 minutes, and immediately sees the error spike started exactly when a new deployment rolled out — pointing straight at the recent release as the root cause, all without writing a single manual query."

13. Splunk Deep Dive — Architecture, Indexing, Forwarders

13.1 What Is Splunk?

Splunk is a commercial, enterprise-grade platform for collecting, indexing, and searching machine-generated data (logs, metrics, events) at massive scale — it plays the same overall role as ELK, but as a mature, polished, paid product with a very powerful proprietary query language (SPL) and strong enterprise support/compliance features.

13.2 Splunk Architecture



13.3 Core Splunk Concepts

Concept	Meaning
Universal Forwarder	Lightweight agent installed on each server/pod that reads log files/stdout and forwards raw data to indexers
Index	Where Splunk stores processed/indexed event data (e.g., <code>main</code> , or a dedicated <code>bank_prod</code> index)
Sourcetype	A classification of the data format (e.g., <code>spring_boot_json</code> , <code>nginx_access</code>) — tells Splunk how to parse fields
Event	One single log entry as stored/indexed in Splunk
Field Extraction	Splunk automatically (or via configured rules) extracts key-value fields from raw events for searching
SPL (Search Processing Language)	Splunk's own query language — pipe-based, similar in feel to Unix shell pipelines

13.4 Getting Logs Into Splunk From a Spring Boot App

```
# inputs.conf on the server/pod running the Universal Forwarder
[monitor:///var/log/accounts-service/*.log]
sourcetype = spring_boot_json
index = bank_prod

# Alternative: Spring Boot logs directly to Splunk HTTP Event Collector (HEC) — common in Kubernetes
<dependency>
  <groupId>com.splunk.logging</groupId>
  <artifactId>splunk-library-javalogging</artifactId>
</dependency>

# log4j2.xml — HEC appender sends logs directly over HTTPS, no forwarder needed
<Http name="splunkHttp" url="https://splunk-hec:8088/services/collector/event"
  token="YOUR-HEC-TOKEN" source="accounts-service" sourcetype="spring_boot_json" index="bank_prod"/>
```

Real scenario: "Our Kubernetes pods run without local disk persistence for logs — instead, each Spring Boot service is configured with a Splunk HEC appender that ships structured JSON logs directly over HTTPS the moment they're written, so we don't lose anything even if a pod crashes and is deleted instantly."

14. Splunk Search Processing Language (SPL) — Real Queries

14.1 Basic Search Syntax

```
# Find all ERROR events from Cards Service in the last 24 hours
index=bank_prod sourcetype=spring_boot_json service="cards-service" level="ERROR"

# Search across a specific time range using earliest/latest
index=bank_prod service="loans-service" earliest=-1h latest=now level="ERROR"
```

14.2 Real SPL Queries Used in Our Organization

```
# 1. Count errors per service in the last hour, sorted highest first
index=bank_prod level="ERROR" earliest=-1h
| stats count by service
| sort -count

# 2. Find the full journey of one customer's request using a trace ID
index=bank_prod traceId="a1b2c3d4"
| sort _time
| table _time, service, level, message

# 3. Top 10 slowest API endpoints in the last 4 hours
index=bank_prod sourcetype=access_log earliest=-4h
| stats avg(durationMs) as avg_duration, p95(durationMs) as p95_duration by endpoint
| sort -p95_duration
| head 10

# 4. Detect a spike in failed logins (possible brute-force attempt)
index=bank_prod sourcetype=auth_log message="Failed login"
| bucket _time span=5m
| stats count by _time, sourceIp
| where count > 20

# 5. Calculate error rate percentage over time for an SLO dashboard
index=bank_prod service="accounts-service"
| bucket _time span=5m
| stats count as total, count(eval(level="ERROR")) as errors by _time
| eval error_rate = round((errors/total)*100, 2)

# 6. Find all audit events where a card limit was changed above a threshold
index=bank_prod sourcetype=audit_log action="card_limit_updated" newLimit>100000
| table _time, user, cardId, oldLimit, newLimit
```

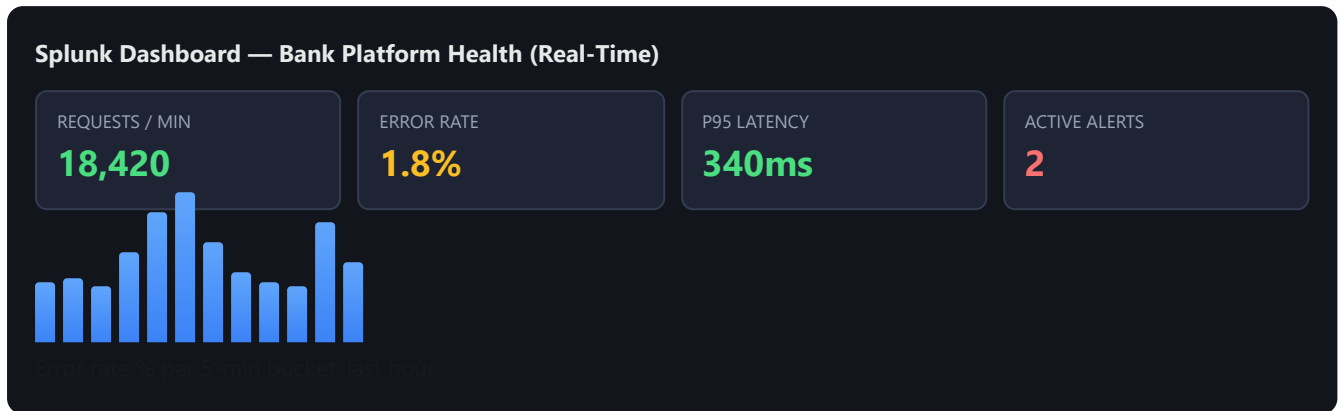
14.3 Key SPL Commands Cheat Sheet

Command	Purpose
stats	Aggregate values (count, avg, sum, p95, etc.) grouped by fields
eval	Create/calculate a new field based on an expression

<code>where</code>	Filter results after aggregation (like SQL's HAVING)
<code>sort</code>	Order results by a field (prefix with <code>-</code> for descending)
<code>table</code>	Display only specific fields in a clean tabular view
<code>bucket</code>	Group events into time buckets (e.g., 5-minute windows) for trend charts
<code>rex</code>	Extract a new field from raw text using a regular expression
<code>timechart</code>	Build a time-series aggregation directly suited for charting

15. Splunk Dashboards — Visual Examples

15.1 Real Splunk Dashboard Mockup — Executive/On-Call View



15.2 Common Panels in Real Splunk Dashboards

- **Single-value panels** — total requests, error rate, active alert count (built with `stats` + a Splunk "Single Value" visualization).
- **Time chart panels** — error trends, traffic trends, built with `timechart`.
- **Top-N tables** — top failing endpoints, top customers affected, slowest queries.
- **Geographic maps** — request origin by country/region for fraud/security dashboards.
- **Alert-linked panels** — panels tied directly to a saved search that also powers a real-time alert.

15.3 Creating a Splunk Alert From a Saved Search

```
# Saved search powering an alert: fire if error rate exceeds 5% for 2 consecutive 5-min windows
index=bank_prod service="accounts-service"
| bucket _time span=5m
| stats count as total, count(eval(level="ERROR")) as errors by _time
| eval error_rate = round((errors/total)*100, 2)
| where error_rate > 5

# Alert configured to: run every 5 minutes, trigger if search returns >=1 result,
# throttle to avoid duplicate pages, action = send to PagerDuty webhook
```

16. ELK vs Splunk vs Kibana — Full Comparison

Aspect	ELK Stack (Elasticsearch + Logstash + Kibana)	Splunk
Cost Model	Open source core (free), paid tier (Elastic subscription) for advanced features	Commercial, licensed by data ingestion volume — can get expensive at scale
Setup/Maintenance	You self-host and maintain the cluster (or use Elastic Cloud)	Can be self-hosted or Splunk Cloud (fully managed)
Query Language	KQL / Elasticsearch Query DSL (JSON-based)	SPL — very powerful, pipe-based, enterprise-favorite
Ease of Use	Good, some learning curve for Query DSL	Very polished UI/UX, often considered more approachable for non-engineers (support/ops teams)
Enterprise Features	Growing (via paid tiers) — security, alerting, ML	Very mature — strong compliance, RBAC, ITSI, enterprise support contracts
Common In	Cloud-native/Kubernetes-heavy, cost-conscious, open-source-friendly orgs	Large enterprises, banks, regulated industries with budget for licensing and strong support needs

Interview tip: Both solve the same core problem (centralized log search/analysis) — the real difference is licensing cost model, operational ownership (self-hosted vs managed), and query language ergonomics. Many large enterprises (including regulated banks) use Splunk specifically for its enterprise compliance/support maturity, which is very likely why your organization uses it.

17. Migrating from Browser/Console Logs to Splunk or ELK (Real Migration Path)

17.1 The Starting Point — "We Just Have Browser Dev Tools / Console Logs"

If today the only visibility into issues is opening browser DevTools Console/Network tab, or SSHing into a server to tail a local log file, you have **zero** centralized observability — nothing is searchable historically, nothing survives a pod restart, and nobody gets alerted automatically when something breaks.

17.2 Step-by-Step Migration Plan

1. **Step 1 — Standardize on structured logging first.** Before shipping logs anywhere, switch every service to JSON structured logging (Section 9) with consistent field names (service, level, traceId, message) — garbage in, garbage out otherwise.
2. **Step 2 — Add Correlation/Trace IDs everywhere.** Ensure every request gets a unique ID propagated through all services and included in every log line (Section 9.3).
3. **Step 3 — Pick the destination.** Confirm Splunk (since your org already has it) as the target — request an index (e.g., `bank_prod`) and an HEC token, or forwarder deployment, from your Splunk admin team.
4. **Step 4 — Instrument one pilot service first.** Don't migrate all 20 services at once — start with one lower-risk service, wire up the Splunk HEC appender or Universal Forwarder, verify events arrive and fields parse correctly.
5. **Step 5 — Build a basic dashboard and one alert for the pilot service.** Prove the value early (e.g., an error-rate dashboard + a Slack/PagerDuty alert) before rolling out further.
6. **Step 6 — Roll out to remaining services incrementally,** reusing the same sourcetype/index conventions and log format standards established in the pilot.
7. **Step 7 — Decommission ad-hoc access** (SSH-based manual log tailing, browser-console-only debugging) once the team trusts and habitually uses Splunk for troubleshooting.
8. **Step 8 — Add distributed tracing** (OpenTelemetry) once basic centralized logging is solid, to complete the observability picture (Sections 20-23).

17.3 What About Frontend (React) Logs Currently Only in Browser DevTools?

Frontend errors visible only in a user's browser console are invisible to your team unless the user reports them. Real organizations solve this by shipping frontend errors/logs to the backend (or directly to an observability tool) using a small client-side error reporting layer.

```
// React – global error boundary + window.onerror reporting to a backend logging endpoint
window.addEventListener('error', (event) => {
  fetch('/api/client-logs', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      message: event.message,
      stack: event.error?.stack,
      url: window.location.href,
      traceId: getCurrentTraceId()
    })
  });
});
```

The backend endpoint receiving these simply logs them (structured JSON, same pipeline as backend logs) into the same Splunk index — so frontend errors become searchable and alertable exactly like backend errors, closing the "we only saw it in dev tools" gap entirely.

18. Application, Infrastructure & Business Metrics

18.1 Categories of Metrics

Category	Examples	Why It Matters
Application Metrics	Request count, response time, error rate per endpoint	Shows how the application itself is performing
Infrastructure Metrics	CPU, memory, disk I/O, network throughput per pod/node	Shows if underlying resources are the bottleneck
Business Metrics	Loan applications/hour, failed payment %, new accounts opened/day	Connects technical health to actual business impact
Custom Metrics	Anything specific to your domain (e.g., "average card approval time")	Captures what generic tooling can't measure out of the box

18.2 Real Custom Business Metric in Spring Boot

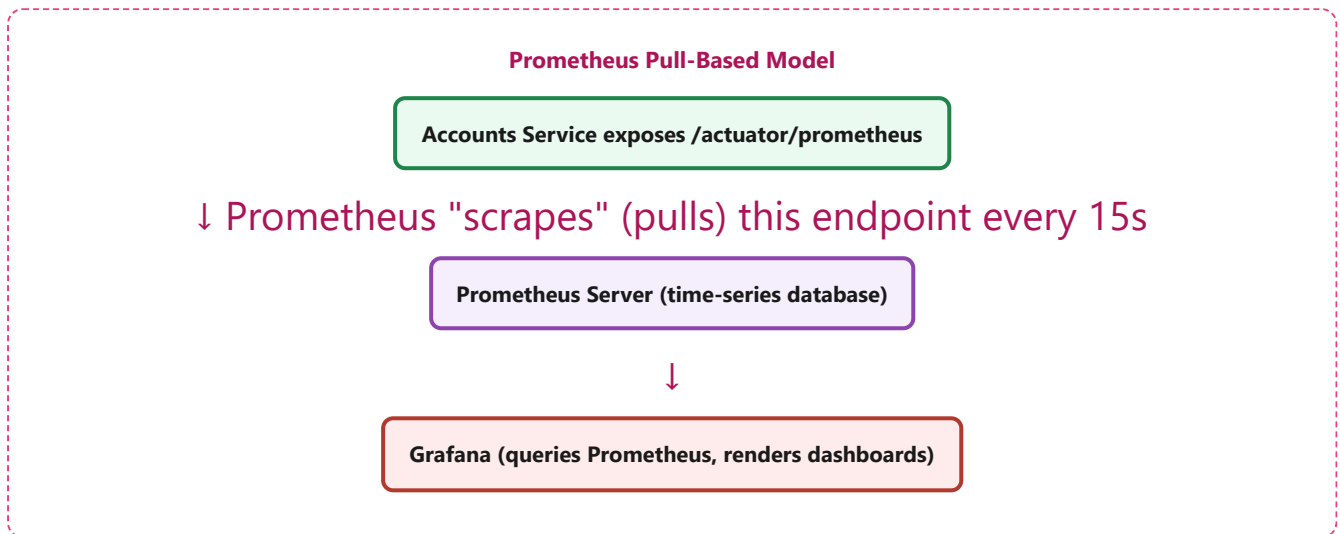
```
@Service
@RequiredArgsConstructor
public class LoansService {
    private final MeterRegistry meterRegistry;

    public void approveLoan(Loan loan) {
        // ... business logic
        meterRegistry.counter("loan.approved", "type", loan.getType()).increment();
        meterRegistry.summary("loan.approved.amount").record(loan.getAmount());
    }
}
```

This one counter powers both a technical dashboard panel ("loan approvals per hour") and a genuinely business-relevant KPI ("are we approving fewer loans than usual today, and why?") — the same metric, two audiences.

19. Prometheus & Grafana Deep Dive with Dashboards

19.1 How Prometheus Works



Unlike log shippers that **push** data, Prometheus typically **pulls** (scrapes) metrics from each target on a schedule — this is a key architectural difference from logging pipelines and something interviewers often ask about.

19.2 Prometheus Configuration (Real Example)

```
# prometheus.yml
scrape_configs:
  - job_name: 'accounts-service'
    metrics_path: '/actuator/prometheus'
    scrape_interval: 15s
    static_configs:
      - targets: ['accounts-service:8081']
  - job_name: 'cards-service'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['cards-service:8082']
```

19.3 PromQL — Prometheus Query Language Examples

```
# Requests per second over the last 5 minutes
rate(http_server_requests_seconds_count{service="accounts-service"}[5m])

# p95 latency
histogram_quantile(0.95, rate(http_server_requests_seconds_bucket[5m]))

# Error rate percentage
sum(rate(http_server_requests_seconds_count{status=~"5.."}[5m]))
/ sum(rate(http_server_requests_seconds_count[5m])) * 100

# JVM heap usage percentage
jvm_memory_used_bytes{area="heap"} / jvm_memory_max_bytes{area="heap"} * 100
```

19.4 Real Grafana Dashboard Mockup

Grafana Dashboard — Accounts Service (RED Metrics)

REQUEST RATE
842 req/s

ERROR RATE
2.1%

P95 DURATION
280ms

HEAP USAGE
68%



19.5 Alerting Rules in Prometheus (Alertmanager)

```
# alert-rules.yml
groups:
- name: accounts-service-alerts
  rules:
  - alert: HighErrorRate
    expr: sum(rate(http_server_requests_seconds_count{status=~"5..",service="accounts-service"}[5m]))
        / sum(rate(http_server_requests_seconds_count{service="accounts-service"}[5m])) > 0.05
    for: 5m
    labels: { severity: critical }
    annotations:
      summary: "Accounts Service error rate above 5% for 5 minutes"
```

20. Spring Boot Observability — Actuator & Micrometer

20.1 Why Actuator Is the Foundation

Spring Boot Actuator exposes production-ready operational endpoints out of the box — health, metrics, environment info — forming the backbone that Kubernetes probes, Prometheus scraping, and Eureka health checks all depend on.

20.2 Key Actuator Endpoints

Endpoint	Purpose
<code>/actuator/health</code>	Overall UP/DOWN status, including DB/Kafka/disk checks
<code>/actuator/health/readiness</code>	Kubernetes readiness probe — "can I accept traffic?"
<code>/actuator/health/liveness</code>	Kubernetes liveness probe — "am I alive at all?"
<code>/actuator/metrics</code>	Browse all raw available metrics
<code>/actuator/prometheus</code>	Metrics formatted for Prometheus scraping
<code>/actuator/loggers</code>	View/change log levels at runtime (Section 4.4)
<code>/actuator/env</code>	Current configuration properties (secure this in prod!)
<code>/actuator/threaddump</code>	Full JVM thread dump — critical for hang/deadlock diagnosis (Section 35)
<code>/actuator/heapdump</code>	Downloadable full heap dump — critical for memory leak diagnosis (Section 34)

20.3 Micrometer — The Metrics Facade

Micrometer is to **metrics** what SLF4J is to **logging** — a vendor-neutral facade. You instrument your code once using Micrometer's API, and it can export to Prometheus, Datadog, New Relic, or others just by swapping a dependency, with no code changes.

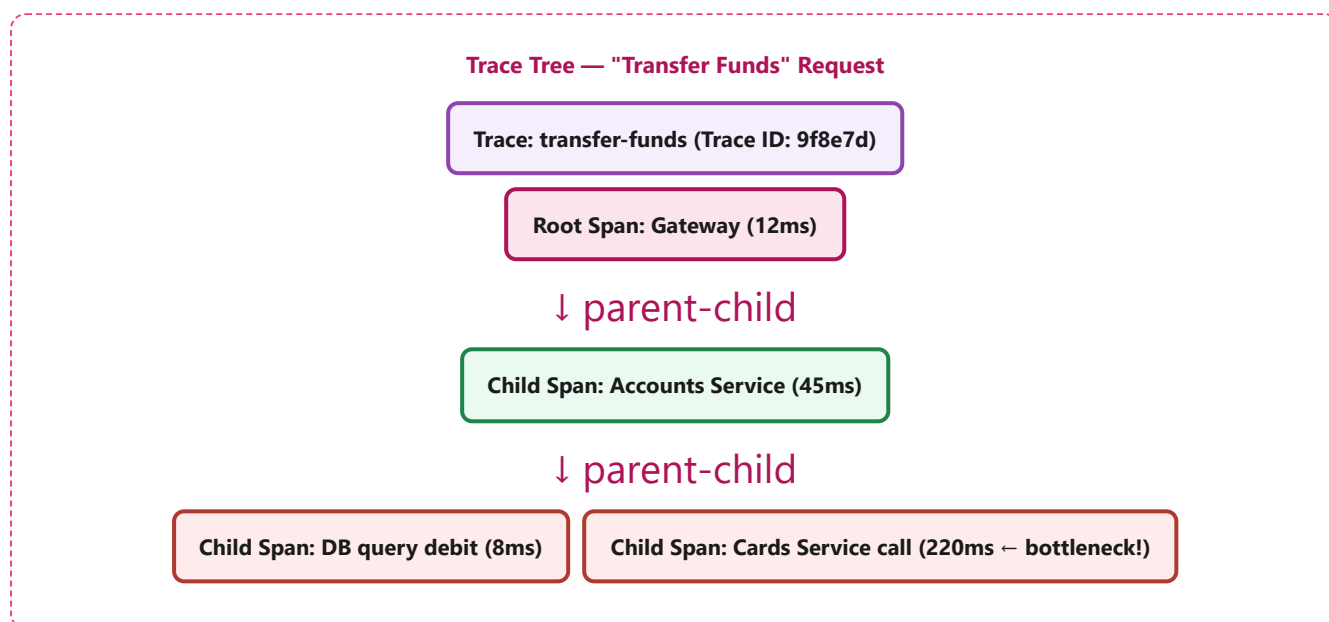
```
management:
  endpoints:
    web:
      exposure:
        include: health, info, prometheus, metrics, loggers, threaddump, heapdump
  endpoint:
    health:
      probes:
        enabled: true      # enables /actuator/health/readiness and /liveness
  metrics:
    tags:
      application: accounts-service  # every metric auto-tagged with service name
```

20.4 Readiness vs Liveness — Real Difference

	Readiness Probe	Liveness Probe
Question Answered	"Can I currently handle traffic?"	"Am I alive, or should I be restarted?"
On Failure	Removed from Service load-balancing (no restart)	Kubernetes kills and restarts the container
Real Example	DB connection pool exhausted — don't send more traffic yet	Deadlocked JVM that will never recover on its own

21. Distributed Tracing — OpenTelemetry, Zipkin, Jaeger

21.1 Trace, Span, Parent Span, Child Span Recap



21.2 OpenTelemetry — The Modern Standard

OpenTelemetry (OTel) is the current CNCF industry standard for generating traces, metrics, and logs in a vendor-neutral way — auto-instruments Spring Boot (HTTP, JDBC, Kafka clients) via a Java agent with zero code changes, and can export to Zipkin, Jaeger, Prometheus, or commercial APM tools interchangeably.

```
java -javaagent:opentelemetry-javaagent.jar \
-Dotel.service.name=accounts-service \
-Dotel.traces.exporter=otlp \
-Dotel.exporter.otlp.endpoint=http://otel-collector:4317 \
-jar accounts-service.jar
```

21.3 Zipkin — Simpler, Lightweight Tracing UI

Originally built at Twitter, Zipkin provides a simple, easy-to-run UI for viewing traces — good for small-to-mid scale setups, easy local development setup via Spring Cloud Sleuth/Micrometer Tracing integration.

```
management:
  tracing:
    sampling:
      probability: 1.0    # trace 100% of requests in dev; lower (e.g., 0.1) in high-traffic prod
zipkin:
  tracing:
    endpoint: http://zipkin:9411/api/v2/spans
```

21.4 Jaeger — Scalable, CNCF Tracing Platform

Originally built at Uber, Jaeger scales further than Zipkin with pluggable storage backends (Elasticsearch, Cassandra), a richer dependency-graph UI, and is a natural fit for larger Kubernetes-native organizations already running an ELK-style storage layer.

21.5 Zipkin vs Jaeger — Quick Comparison

Aspect	Zipkin	Jaeger
Origin	Twitter	Uber (CNCF graduated project)
Storage backend options	In-memory, MySQL, Elasticsearch (simpler)	Elasticsearch, Cassandra, Kafka-buffered (more scalable)
UI richness	Simple, functional	Richer — service dependency graphs, more advanced filtering
Best fit	Small-mid scale, simpler setups	Large-scale, Kubernetes-native, CNCF-aligned organizations

21.6 Root Cause Analysis Using Tracing — Real Walkthrough

1. Grafana alert fires: Accounts Service p95 latency > 1s.
2. Open Jaeger, filter traces for Accounts Service in that time window, sort by duration descending.
3. Open the slowest trace — see the span breakdown (Gateway → Accounts → Cards call → DB query).
4. Spot that the "Cards Service call" span consistently takes 200-300ms, dominating total request time.
5. Cross-reference Cards Service's own logs (via the same trace ID in Splunk/Kibana) to find the actual root cause (e.g., a slow downstream Card Network API call without a timeout).

22. Monitoring Types — JVM, CPU, Memory, Disk, Network, Database, API

22.1 JVM Monitoring

Metric	Why It Matters
Heap usage (used/max)	Approaching the max risks OutOfMemoryError
GC pause time & frequency	Long/frequent GC pauses directly cause request latency spikes
Thread count	Runaway thread creation indicates a leak or thread pool misconfiguration
Class loading count	Unusual growth can indicate classloader leaks (common in app server redeloys)

22.2 CPU, Memory, Disk, Network Monitoring

Resource	What to Watch	Real Consequence If Ignored
CPU	Sustained utilization %, throttling events (Kubernetes CPU limits)	Request queueing, increased latency, eventual timeouts
Memory	Usage vs limit, OOMKilled events	Container abruptly killed and restarted, dropping in-flight requests
Disk	Free space %, I/O wait time	Application/log writes fail once disk is full, cascading failures
Network	Bandwidth usage, connection errors, DNS resolution time	Intermittent timeouts hard to diagnose without network-level metrics

22.3 Database Monitoring

- Connection pool usage (active vs idle vs max) — exhaustion causes new requests to wait/fail.
- Slow query log / query duration percentiles (Section 36).
- Replication lag (for read replicas) — stale reads if lag grows.
- Lock waits/deadlocks — a common hidden cause of intermittent timeouts.

22.4 API Monitoring

- Per-endpoint request rate, error rate, latency percentiles (not just service-wide averages — one bad endpoint can hide in a good overall average).
- Status code breakdown (4xx vs 5xx) — 4xx spikes often indicate a client-side/contract issue, not a backend outage.
- Rate limit rejection counts — helps distinguish "we're down" from "a client is being throttled as designed."

23. APM Tools — Dynatrace, AppDynamics, New Relic

23.1 What Full APM Platforms Add Beyond Open Source Tooling

Application Performance Monitoring (APM) platforms like Dynatrace, AppDynamics, and New Relic combine metrics, distributed tracing, log aggregation, real-user monitoring (RUM), and AI-driven root-cause analysis into one commercial, fully managed product — reducing the operational burden of running/integrating Prometheus + Grafana + Jaeger + ELK yourself.

23.2 Comparison

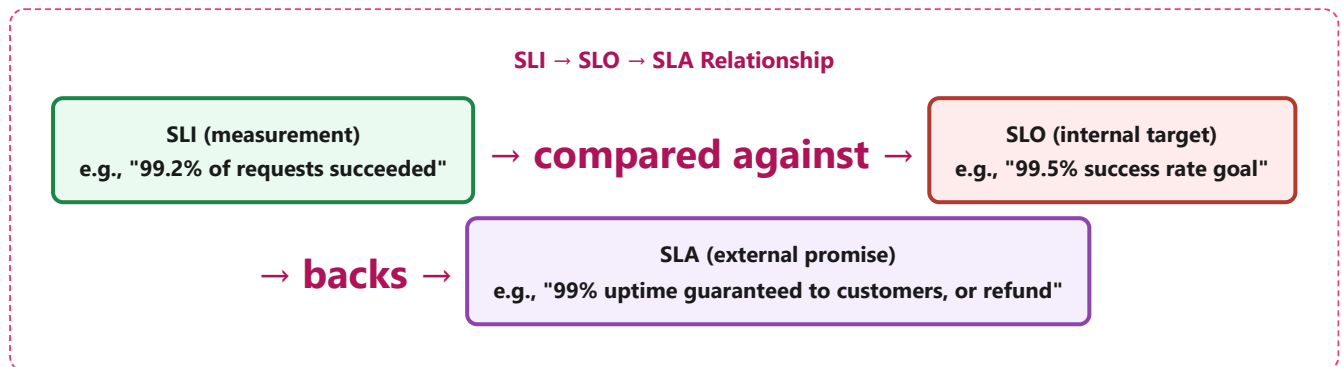
Tool	Standout Feature	Best Fit
Dynatrace	AI-powered automatic root-cause analysis ("Davis AI"), full-stack auto-discovery	Large enterprises wanting minimal manual dashboard/alert configuration
AppDynamics	Strong business transaction correlation (ties technical metrics to business KPIs)	Enterprises wanting tight technical-to-business impact visibility
New Relic	Broad, approachable all-in-one platform, generous free tier for smaller teams	Teams wanting a single commercial tool without heavy self-hosting

23.3 Open Source Stack vs Commercial APM — Trade-off

Aspect	Open Source (Prometheus/Grafana/Jaeger/ELK)	Commercial APM (Dynatrace/AppDynamics/New Relic)
Cost	Free/self-hosted infra cost only	Licensing cost, often per-host/per-GB, can scale expensively
Setup effort	You wire everything together yourself	Mostly auto-instrumented/auto-discovered out of the box
Flexibility	Full control, fully customizable	Some customization, but within the vendor's platform constraints
Best for	Cost-conscious, engineering-heavy orgs wanting full control	Enterprises prioritizing fast time-to-value and reduced operational overhead

24. Alerting — SLA, SLO, SLI, Alertmanager, PagerDuty

24.1 SLI, SLO, SLA — The Relationship



Term	Meaning	Real Example
SLI (Service Level Indicator)	The actual measured metric	"99.2% of requests returned successfully last month"
SLO (Service Level Objective)	The internal target/goal for that metric	"We aim for 99.5% success rate"
SLA (Service Level Agreement)	The external, often contractual promise to customers, usually with a looser target than the internal SLO (as a safety buffer) and consequences if missed	"99% uptime guaranteed, or service credit issued"

Interview tip: SLOs are always set stricter than the public SLA on purpose — this gives the team an internal early-warning buffer to react before ever actually breaching the customer-facing SLA.

24.2 Error Budgets

If your SLO is 99.9% success rate, your "error budget" for the month is the remaining 0.1% — a practical way to balance reliability work vs feature velocity: if the error budget is being consumed too fast, the team pauses risky releases and focuses on stability.

24.3 Alertmanager — Routing & Silencing Prometheus Alerts

```
# alertmanager.yml
route:
  receiver: 'team-accounts-slack'
  routes:
    - match: { severity: critical }
      receiver: 'pagerduty-oncall'
      continue: true
receivers:
  - name: 'pagerduty-oncall'
    pagerduty_configs:
      - routing_key: '<PAGERDUTY_INTEGRATION_KEY>'
  - name: 'team-accounts-slack'
    slack_configs:
      - channel: '#accounts-service-alerts'
```

24.4 Grafana Alerts (Alternative/Complementary to Alertmanager)

Grafana's built-in alerting lets you define alert rules directly on dashboard panels (e.g., "alert if this panel's value exceeds threshold for 5 minutes"), simplifying the setup when you're already building dashboards in Grafana rather than maintaining separate Alertmanager rule files.

24.5 PagerDuty — Incident Notification & Escalation

- Receives alerts from Prometheus/Alertmanager, Splunk, Grafana, or any webhook source.
- Manages on-call schedules/rotations per team.
- Escalates automatically if the primary on-call doesn't acknowledge within a configured time.
- Tracks incident timeline (acknowledged, resolved) for post-incident reviews.

24.6 Good Alerting Practices

Practice	Why
Alert on symptoms, not causes	"Error rate high" (symptom, user-impacting) is better than "CPU at 80%" (cause, may not even matter to users)
Use "for" duration, not instant spikes	Avoids paging someone for a 10-second blip that self-resolves
Tie alerts to actionable runbooks	Every alert should link to documented next steps, not leave the on-call guessing
Avoid alert fatigue	Too many low-value alerts cause real ones to get ignored/muted — a serious anti-pattern

25. Azure Monitor & Azure DevOps Integration for Logging and Monitoring

25.1 Azure-Native Observability Services

Azure Service	Role
Azure Monitor	The umbrella platform collecting metrics and logs across all Azure resources (AKS, App Service, databases)
Log Analytics Workspace	Where Azure Monitor logs are stored/queried, using Kusto Query Language (KQL)
Application Insights	APM specifically for applications — auto-instruments requests, dependencies, exceptions, and can capture distributed traces
Container Insights	AKS-specific monitoring — pod/node CPU, memory, and log collection built on Log Analytics
Azure Managed Grafana / Managed Prometheus	Fully managed Prometheus + Grafana offering, if you prefer that stack over Azure Monitor's native tools

25.2 Application Insights — Real Spring Boot Integration

```
<dependency>
  <groupId>com.microsoft.azure</groupId>
  <artifactId>applicationinsights-spring-boot-starter</artifactId>
</dependency>

# application.yml
azure:
  application-insights:
    connection-string: InstrumentationKey=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx
```

Once wired up, Application Insights automatically captures incoming requests, outgoing dependency calls (DB, HTTP, Kafka), exceptions, and custom events/metrics — visualized in the Azure Portal with a built-in "Application Map" showing your live service dependency graph.

25.3 Kusto Query Language (KQL) — Real Examples

```
// Find all exceptions in the last hour for Accounts Service
exceptions
| where cloud_RoleName == "accounts-service"
| where timestamp > ago(1h)
| project timestamp, type, outerMessage, operation_Id

// Requests slower than 2 seconds, grouped by operation name
requests
| where duration > 2000
| summarize count(), avg(duration) by name
| order by count_desc
```

25.4 How This Fits With Azure DevOps (Since Your Org Uses It)

- **Release annotations:** Azure DevOps pipeline deployment steps can push a "release marker" into Application Insights/Azure Monitor, so dashboards visually show exactly when a new version was deployed — instantly correlating incidents with recent releases.
- **Pipeline-triggered smoke test alerts:** a post-deployment pipeline stage can query Application Insights/Log Analytics for error rate immediately after deployment, and automatically roll back if it spikes.
- **Work item linking:** alerts/incidents from Azure Monitor can automatically create linked Azure Boards work items for tracking and follow-up.
- **Unified secrets/config:** Application Insights connection strings and Splunk HEC tokens are both stored in Azure Key Vault-backed Variable Groups, injected consistently across all pipelines.

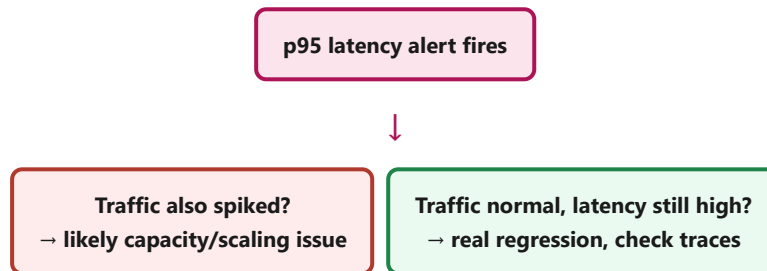
Real scenario: "Our Azure DevOps release pipeline for Accounts Service automatically annotates our Grafana/Application Insights dashboard with a vertical marker on every production deployment — when an on-call engineer sees an error spike, the first thing they check is whether it lines up with one of these markers, immediately narrowing down 'is this a bad deploy or something else.'"

26. Real-Time Production Troubleshooting (Response Time, Memory Leak, CPU, Thread Dump, GC, Slow SQL)

26.1 High Response Time Analysis

Step-by-step: (1) Check Grafana/Splunk dashboard — is it every endpoint or just one? (2) Check the Golden Signal panels — is traffic unusually high (real load) or is latency high even at normal traffic (a real regression)? (3) Open a distributed trace for a slow request — find which span dominates. (4) Drill into that specific component (DB query, downstream call, GC pause).

High Response Time — Decision Flow



26.2 Memory Leak Detection

A memory leak shows as a heap usage graph that keeps climbing over hours/days and never drops back down even after full GC cycles — eventually leading to OutOfMemoryError or repeated OOMKills.

```
# Step 1: confirm the trend in Grafana – steadily rising heap usage over days despite GC runs
# Step 2: capture a heap dump for analysis (via Actuator or jmap)
curl -o heapdump.hprof http://accounts-service:8081/actuator/heapdump
jmap -dump:live,format=b,file=heapdump.hprof <pid>

# Step 3: analyze with Eclipse MAT (Memory Analyzer Tool) – look at "Leak Suspects" report
# common real culprits: unbounded static caches/Maps, listeners never unregistered, ThreadLocal not cleared
```

Real scenario: "A slow, multi-day memory creep in Cards Service was traced (via Eclipse MAT's dominator tree) to a static `Map<Long, CardDto>` cache that was never evicted — every unique card ID ever looked up stayed in memory forever. Fixed by switching to a bounded Caffeine cache with a TTL and max size."

26.3 High CPU Investigation & Thread Dump Analysis

```
# Step 1: identify the high-CPU JVM process on the host/pod
top -H -p <pid>    # shows per-thread CPU usage

# Step 2: capture a thread dump at that moment
curl http://accounts-service:8081/actuator/threadsdump > threaddump.json
jstack <pid> > threaddump.txt

# Step 3: convert the high-CPU thread's native ID (from top -H) to hex, find matching "nid=0x..." in the thread dump
# Step 4: inspect that thread's stack trace to see exactly what code it's stuck executing
```

Real scenario: "High sustained CPU on Loans Service was traced via thread dump to a regex pattern with catastrophic backtracking, running on every single interest calculation request — visible as dozens of threads all stuck inside the same `Pattern.matches()` call in their stack traces."

26.4 GC (Garbage Collection) Analysis

```
# Enable GC logging (Java 9+)
-Xlog:gc*:file=/logs/gc.log:time,uptime:filecount=10,filesize=50M

# Analyze with GCViewer or GCEasy.io – look for:
# - Frequent Full GCs (should be rare)
# - Long pause times (each pause = frozen application, no requests processed)
# - Rising "after GC" heap baseline over time (memory leak indicator)
```

Symptom	Likely Cause	Fix Direction
Frequent minor GCs	Too much short-lived object churn (young gen too small)	Tune young generation size, reduce unnecessary object allocation
Long Full GC pauses	Old generation filling up, or memory leak	Increase heap, fix leak, consider G1GC/ZGC tuning
Heap baseline keeps rising after each GC	Memory leak — objects not being released	Heap dump analysis (Section 26.2)

26.5 Slow SQL Analysis

```
# Step 1: Enable slow query logging on the database (MySQL example)
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 2;    # log queries taking more than 2 seconds

# Step 2: identify the actual slow query and run EXPLAIN on it
EXPLAIN SELECT * FROM transaction WHERE account_id = 1001 AND created_date > '2026-01-01';

# Step 3: check if an index is missing (look for "type: ALL" = full table scan in EXPLAIN output)
CREATE INDEX idx_txn_account_date ON transaction(account_id, created_date);
```

Real scenario: "A 'random' slowness on the Accounts Service dashboard was traced through a distributed trace's DB-query span, then to the slow query log, revealing a missing composite index on `transaction(account_id, created_date)` —

a query that once took 4.5 seconds dropped to 12ms after adding the index."

27. Real Organization Setup — End to End (Our Application)

27.1 The Full Observability Stack We Actually Run



27.2 What Each Layer Gives Us (Mapped to the Three Pillars)

Pillar	Our Tool	Primary Use
Logs	Splunk	Deep-diving one request/incident, audit trail, security investigation
Metrics	Azure Monitor / Prometheus + Grafana	Dashboards, trend analysis, alerting on Golden Signals
Traces	Application Insights (or Jaeger, depending on service)	Pinpointing exactly which hop in a multi-service call is slow/failing

27.3 Real Incident Walkthrough Using the Full Stack

1. PagerDuty pages the on-call engineer: "Accounts Service error rate > 5% for 5 minutes."
2. Open Grafana dashboard — confirm the error rate panel and check if it aligns with a recent Azure DevOps deployment marker.
3. Open Application Insights, find a failing request's trace — see it fails inside a call to Cards Service.
4. Search Splunk using the trace/operation ID — find the exact exception logged by Cards Service: a downstream Card Network API timeout.

5. Check the Circuit Breaker's state (Resilience4j metrics exposed via Actuator/Micrometer) — confirm it's open, meaning the fallback is already active and degrading gracefully.
6. Escalate to the third-party Card Network's status page/support, while the fallback keeps most functionality working; document in the incident timeline for the post-incident review.

28. Common Observability Issues, Problems & Solutions

Issue	Cause	Real Solution
"We only found out about the outage from a customer complaint"	No alerting configured, or alerts not tied to actual user-facing symptoms	Set up Golden Signal alerts (error rate, latency) tied to PagerDuty, tested regularly
Can't find logs for a specific failed request	No correlation/trace ID propagated through all services	Implement Correlation ID filter (Section 9.3) across every service and the frontend
Log volume/cost exploding	DEBUG/TRACE level left on in production, or logging entire request/response bodies	Enforce INFO as default production level, sample verbose logs, avoid logging full payloads
Sensitive data found in logs (PII/PCI)	No log masking/redaction policy	Redact card numbers/passwords/tokens before logging; automated scanning for accidental leaks
Alert fatigue — team ignoring alerts	Too many low-value/noisy alerts, no clear runbooks	Alert only on user-impacting symptoms, tune thresholds, attach runbooks to every alert
Dashboards show "everything is fine" during an actual outage	Metrics based on averages hiding a subset of failing requests/endpoints	Use percentiles (p95/p99) and per-endpoint breakdowns, not just overall averages
Can't tell which service is the bottleneck in a slow multi-service request	No distributed tracing in place	Instrument with OpenTelemetry/Application Insights, visualize in Jaeger/Zipkin
Frontend errors invisible to the team	Errors only appear in the user's browser console, never reported anywhere	Ship client-side errors to a backend logging endpoint (Section 17.3)
Splunk/ELK ingestion costs spiraling	Logging too verbosely, duplicate logs, unindexed noisy fields	Right-size log levels, filter out noisy low-value logs at the source/forwarder, review retention policy
Memory leak only shows up after days in production, not in testing	Load/duration in test environments too short to reveal slow leaks	Run longer-duration soak tests in staging, monitor heap trend over multi-day windows before major releases
Different teams use inconsistent log formats/field names	No shared logging standard across services	Establish and enforce an org-wide structured logging standard (field names, correlation ID header) via a shared starter library

29. Interview Questions & Answers (Basic → Advanced → Scenario Based)

Q1. Why do we need logging in an application?

Because production applications run unattended 24/7 — logging provides a persistent, searchable record of what happened, essential for debugging issues after the fact, tracking business events, security auditing, and performance analysis, none of which console output alone can provide.

Q2. What is the difference between monitoring and observability?

Monitoring watches predefined metrics/dashboards for known failure conditions ("is the system healthy?"). Observability provides rich enough telemetry (logs, metrics, traces) to investigate and answer new, previously unanticipated questions about "why" something is happening.

Q3. What are the three pillars of observability?

Logs (detailed event records — "what exactly happened"), Metrics (aggregated numeric trends — "how is the system trending"), and Traces (the end-to-end path of one request across services — "where did time go for this specific request").

Q4. What are the four Golden Signals?

Latency (how long requests take), Traffic (demand on the system), Errors (rate of failures), and Saturation (how full/stressed the system's resources are) — the minimum set of metrics recommended for monitoring any user-facing service.

Q5. What is a Correlation ID and why is it important?

A unique identifier generated at the start of a request and propagated through every downstream service call and log line, letting you find every log related to one specific request across an entire distributed system with a single search.

Q6. Difference between Trace ID and Span ID?

A Trace ID identifies the entire end-to-end request across all services. A Span ID identifies one specific unit of work within that trace (e.g., one service's handling of the request, or one DB query) — spans are linked in a parent-child hierarchy to reconstruct the full call tree.

Q7. What is SLF4J, and why isn't it a logging framework itself?

SLF4J is a facade/API — your code depends on SLF4J's interfaces, while the actual logging implementation (Logback, Log4j2) is plugged in underneath. This lets you switch the underlying logging engine without changing application code.

Q8. Why is DEBUG-level logging risky to leave enabled in production?

It generates a very high volume of log data, increasing storage/ingestion cost (especially with paid tools like Splunk billed on volume), can slow down the application under high load, and risks accidentally logging sensitive data not meant for verbose output.

Q9. How does Prometheus collect metrics, and how is that different from log shipping?

Prometheus uses a pull model — it scrapes a metrics endpoint (like Spring Boot Actuator's /actuator/prometheus) on a schedule. Log shippers (Filebeat, Splunk forwarders) typically push data as it's generated — a key architectural difference often asked about in interviews.

Q10. Explain SLI, SLO, and SLA with an example.

SLI is the actual measured value (e.g., "99.2% success rate this month"). SLO is the internal target (e.g., "we aim for 99.5%"). SLA is the external, often contractual promise to customers (e.g., "99% uptime guaranteed"), deliberately set looser than the SLO to give the team a safety buffer.

Q11. How would you investigate a memory leak in a Spring Boot application in production?

Confirm a steadily rising heap trend in Grafana that doesn't drop after GC, capture a heap dump via Actuator's `/actuator/heapdump` or `jmap`, and analyze it with Eclipse MAT's "Leak Suspects" report to find what's holding references — commonly unbounded caches, unregistered listeners, or `ThreadLocal` misuse.

Real scenario: a static `Map` cache that was never evicted caused a multi-day memory creep, fixed by switching to a bounded, TTL-based `Caffeine` cache.

Q12. How would you diagnose high CPU usage on a production JVM?

Use ``top -H -p <pid>`` to find the specific high-CPU thread, capture a thread dump at that moment (Actuator's `/actuator/threaddump` or `jstack`), convert the thread's native ID to hex, and locate its exact stack trace to see what code it's stuck executing.

Q13. What's the difference between Zipkin and Jaeger?

Both are distributed tracing UIs. Zipkin is simpler and lighter, good for smaller scale. Jaeger (from Uber, CNCF) scales further with more storage backend options and a richer service dependency graph UI, making it a common choice for larger Kubernetes-native organizations.

Q14. What is the difference between ELK and Splunk?

Both centralize and search logs at scale. ELK is open-source (with paid tiers) and self-hosted/self-managed by default; Splunk is a commercial product (or Splunk Cloud) with a very powerful SPL query language, generally considered more polished/enterprise-ready but licensed by ingestion volume, which can get expensive at scale.

Q15. How would you migrate an application currently relying only on browser console logs to a centralized tool like Splunk?

Standardize on structured JSON logging with correlation IDs first, pick a pilot service, wire it to Splunk via HEC or a Universal Forwarder, validate field parsing and build one dashboard/alert to prove value, then roll out incrementally to remaining services — and separately, add a client-side error handler in the frontend that ships browser errors to a backend logging endpoint so they land in the same centralized system.

Q16. What is the purpose of readiness and liveness probes, and how do they relate to Actuator?

Readiness (`/actuator/health/readiness`) tells Kubernetes if a pod should currently receive traffic — failing it removes the pod from load balancing without restarting it. Liveness (`/actuator/health/liveness`) tells Kubernetes if the pod is alive at all — failing it triggers a container restart.

Q17. How do you find the root cause of a slow multi-service request?

Use distributed tracing to find the specific slow span in the call chain (e.g., via Jaeger or Application Insights), then cross-reference centralized logs (Splunk/Kibana) filtered by the same trace ID to see the detailed context/error for that exact slow component.

Q18. What should you never log, and why?

Passwords, full card numbers, JWT tokens, and other sensitive PII/PCI data — logging them in plain text creates serious compliance risk (PCI-DSS/GDPR) and a major security exposure if logs are ever leaked or accessed by unauthorized users; mask/redact sensitive fields before logging.

Q19. What is an error budget and how is it used?

The allowed amount of failure implied by an SLO (e.g., a 99.9% SLO implies a 0.1% error budget). Teams use it to balance reliability and feature velocity — if the budget is being consumed too fast, the team pauses risky releases to focus on stability instead.

Q20. Walk through a real production incident you resolved using logs, metrics, and traces together.

An alert fired on high error rate (metrics); a distributed trace showed the failure occurred in a downstream Cards Service call (traces); searching centralized logs by that request's trace ID revealed the exact exception — a third-party Card Network API timeout (logs) — while the circuit breaker's fallback kept the system functioning in a degraded but stable state during the incident.

30. Final Summary & Cheat Sheet

The journey: `System.out.println()` → Log4j/SLF4J/Logback with levels and files → Centralized logging (Splunk/ELK) with structured JSON and correlation IDs → Full observability (Logs + Metrics + Traces) with Prometheus/Grafana, OpenTelemetry/Jaeger/Zipkin, and Actuator/Micrometer.

Our real stack: Splunk for centralized log search and SPL-powered dashboards/alerts, Azure Monitor/Prometheus+Grafana for metrics, Application Insights/Jaeger for distributed tracing, Alertmanager/PagerDuty for incident notification — all integrated with our Azure DevOps pipelines for deployment-correlated observability.

The golden rule: Instrument for observability from day one (structured logs, correlation IDs, health probes, basic metrics) rather than bolting it on after the first painful production incident where nobody could figure out what went wrong.

30.1 Quick Cheat Sheet

Term	One-Line Meaning
SLF4J	Logging facade — code against this, swap the implementation freely
Log Level	TRACE < DEBUG < INFO < WARN < ERROR — controls verbosity
Correlation ID	Unique ID tying together all logs for one request across services
Trace / Span	Trace = whole request journey; Span = one timed unit of work within it
Golden Signals	Latency, Traffic, Errors, Saturation
ELK	Elasticsearch (store/search) + Logstash (process) + Kibana (visualize)
Splunk	Commercial log platform with SPL query language, forwarders, and HEC ingestion
Prometheus	Pull-based metrics time-series database
Grafana	Dashboards and alerting on top of Prometheus (or other) data sources
Micrometer	Metrics facade for Spring Boot, vendor-neutral like SLF4J for metrics
Actuator	Spring Boot's built-in health/metrics/ops endpoints
OpenTelemetry	Vendor-neutral standard for generating traces/metrics/logs
SLI / SLO / SLA	Measured value / internal target / external contractual promise
Error Budget	Allowed failure margin implied by an SLO, balances velocity vs stability
Heap Dump	Snapshot of JVM memory used to diagnose memory leaks
Thread Dump	Snapshot of all JVM threads used to diagnose hangs/high CPU/deadlocks

Use this guide alongside the Git/GitHub/GitLab/Bitbucket guide, the CI/CD guide, the Docker & Kubernetes guide, and the Microservices guide — together they cover the complete real-world Java full-stack developer journey: writing and versioning code, automating build/test/deploy, containerizing and orchestrating services, designing resilient microservices, and finally, observing and troubleshooting all of it reliably in production.